

Clase Septiembre 28 de 2026 — Kernel (resuelto)

Materia: Sistemas Operativos, UAO **Fuente:** ClaseSEPTIEMBRE2826.pdf (41 diapositivas sobre el kernel)

Del PDF saqué tres cosas que hay que resolver:

1. **Actividad de laboratorio (diapositivas 38-39):** identificar el sistema FreeBSD y compilar e instalar un kernel personalizado.
2. **Trabajo investigativo (diapositiva 40), entrega el 5 de octubre:** presentar un sistema operativo por cada tipo de kernel.
3. **Versión 6 (diapositiva 40), entrega el 19 de octubre:** instalar un kernel nuevo en Oracle Linux, Ubuntu o CentOS, y clasificar el kernel de varios SO.

Antes de resolver, dejo un resumen de la teoría de la clase. Me sirvió para entender las respuestas y no solo copiarlas.

0. Resumen de la teoría (para tener contexto)

- **Kernel:** es el núcleo del SO. Administra el hardware, sobre todo la **CPU, la memoria y los dispositivos de E/S**. Además da protección (niveles de acceso) y acceso compartido (multiplexado) a los recursos.
- **Modo kernel y modo usuario:** la CPU tiene niveles de privilegio (*rings*). Las aplicaciones corren en espacio de usuario y le piden cosas al kernel con **llamadas al sistema (syscalls)**.
- **Kernel panic:** error grave que el kernel detecta y no puede resolver, por ejemplo una referencia a una dirección de memoria inválida. Puede venir de un bug o de una falla de hardware como la RAM.
- **Versiones del kernel Linux (XX.YY.ZZ):**
 - XX es la serie principal.
 - YY par significa producción (estable) e impar significa desarrollo.
 - ZZ es la revisión que corrige bugs.
 - Ejemplo: 2.4.0 es serie 2, producción 4 (par), primera versión. 2.5.0 es la versión 0 del kernel de desarrollo 2.5.
- **Módulos:** en Linux son los `.ko` que están en `/lib/modules`. Se manejan con `lsmod`, `modprobe`, `modinfo` y `depmod`. En FreeBSD están en `/boot/kernel` y se cargan con `kldload`.
- **Por qué compilar un kernel propio (FreeBSD):**
 - Arranca más rápido.
 - Usa menos memoria.
 - Permite soportar hardware que GENERIC no incluye.

1. Actividad de laboratorio: FreeBSD

1.1 Identificar el sistema (diapositiva 38)

Estos tres comandos me dicen con qué sistema estoy trabajando antes de tocar el kernel.

```
cat /etc/os-release
freebsd-version -u -k -r
getconf LONG_BIT
```

Paso a paso:

1. `cat /etc/os-release` muestra el nombre y la versión de la distribución, en este caso FreeBSD. Sirve para confirmar el SO.
2. `freebsd-version -u -k -r` imprime tres versiones:
3. `-u` es la versión de **userland**, o sea las herramientas del espacio de usuario.
4. `-k` es la versión del **kernel instalado**.
5. `-r` es la versión del **kernel que está corriendo** ahora.

Si `-k` y `-r` son distintas, instalé un kernel nuevo y todavía no reinicié. Esto lo voy a usar al final para comprobar que la instalación funcionó. 3. `getconf LONG_BIT` devuelve 32 o 64. Me dice la arquitectura de bits, y eso decide la carpeta de configuración del kernel (ver la nota del paso 1).

1.2 Configurar el kernel (diapositiva 39)

```
cd /usr/src/sys/i386/conf
mkdir /root/kernels
cp GENERIC /root/kernels/CLASEKERN
ln -s /root/kernels/CLASEKERN
```

Paso a paso:

1. `cd /usr/src/sys/i386/conf` entra a la carpeta donde vive el archivo de configuración del kernel. Todo el código del kernel está en `/usr/src/sys`.
2. **Ojo:** `i386` es para 32 bits. Si `getconf LONG_BIT` dio 64, la ruta es `/usr/src/sys/amd64/conf`. La diapositiva 37 explica que el directorio depende de la arquitectura.
3. Si `/usr/src/sys` no existe, no están instaladas las fuentes del kernel y hay que instalarlas primero (la diapositiva 37 lo explica con `sysinstall`; en versiones nuevas se instalan con el `set src`).
4. `mkdir /root/kernels` crea una carpeta **fuera** del árbol de fuentes para guardar mi configuración. Así una actualización de `/usr/src` no la borra.
5. `cp GENERIC /root/kernels/CLASEKERN` copia la configuración genérica, que soporta mucho hardware, con el nombre de mi kernel. `CLASEKERN` es el nombre que usaré al compilar. Aquí es donde se personaliza: se quitan los dispositivos que no tengo y se dejan solo los necesarios (línea `cpu`, `machine`, etc.).
6. `ln -s /root/kernels/CLASEKERN` crea un **enlace simbólico** en la carpeta actual (`.../conf`) apuntando a mi archivo. Así `make` encuentra `CLASEKERN` en `conf/`, pero el archivo real queda protegido en `/root/kernels`.

1.3 Compilar e instalar (diapositiva 39)

```
cd /usr/src
make buildkernel KERNCONF=CLASEKERN
make installkernel KERNCONF=CLASEKERN
init 6
```

Paso a paso:

1. `cd /usr/src` es necesario porque el Makefile que gestiona la compilación está ahí.
2. `make buildkernel KERNCONF=CLASEKERN` compila el kernel usando mi archivo de configuración. Tarda bastante, y la diapositiva 35 avisa que es un proceso largo. No toca el sistema en uso, solo genera los archivos.
3. `make installkernel KERNCONF=CLASEKERN` copia el kernel compilado a `/boot/kernel`. FreeBSD conserva el anterior en `/boot/kernel.old`, así que hay respaldo si algo sale mal.
4. `init 6` reinicia el sistema (cambia al runlevel 6). Es equivalente a `shutdown -r now`. Al arrancar se usa el kernel nuevo.

Cosa que descubrí al hacerlo de verdad: en FreeBSD 15.1 el kernel que viene instalado es de paquetes (pkgbase), y `make installkernel` se niega a pisarlo y da error. La solución es instalar el kernel nuevo en otra ruta con `INSTKERNNAME`, y así el kernel GENERIC queda intacto como respaldo:

```
make installkernel KERNCONF=CLASEKERN INSTKERNNAME=CLASEKERN
nextboot -k CLASEKERN
init 6
```

`nextboot -k` le dice al cargador que arranque CLASEKERN solo en el siguiente reinicio. Si algo falla, el siguiente arranque vuelve solo al kernel GENERIC.

1.4 Verificación después de reiniciar

```
uname -a
freebsd-version -k -r
sysctl kern.conf.txt | head
```

`uname -a` debería mostrar CLASEKERN al final de la cadena del kernel, y `sysctl kern.bootfile` debería apuntar a `/boot/CLASEKERN/kernel`.

Si falla el arranque: en el menú de arranque de FreeBSD se puede elegir otro kernel para volver al anterior (`kernel.old` en una instalación clásica, o `kernel` cuando se usó `INSTKERNNAME`).

2. Trabajo investigativo (entrega individual: 5 de octubre de 2026)

Piden presentar **un sistema operativo por cada tipo de kernel**. Mi elección, con la razón de cada una:

Tipo de kernel / diseño	SO que presento	Por qué encaja
Kernel monolítico	Linux	Todos los servicios (planificador, memoria, sistema de archivos, drivers, red) corren en un solo espacio de kernel. Es rápido porque todo se llama directamente, pero un fallo en un driver puede tumbar el sistema.
Kernel con diseño modular	Linux (LKM) o FreeBSD (kldload)	Es un kernel que carga y descarga módulos (.ko) en caliente sin recompilar. Lo vimos con lsmod, modprobe y kldload.
Sistema estructurado en capas	THE (Dijkstra, 1968)	Tiene capas numeradas, y cada una solo usa los servicios de la capa inferior. La ventaja es que es fácil depurar y verificar capa por capa. La desventaja es el costo de atravesar capas.
Sistema con micronúcleo	MINIX 3 (también QNX)	El kernel solo hace lo mínimo: procesos, memoria básica y comunicación entre procesos (IPC). Drivers y sistema de archivos corren como procesos de usuario. Es más robusto, pero la comunicación por mensajes cuesta rendimiento.
Máquina virtual	IBM VM/370 (z/VM)	Un monitor de máquina virtual crea copias virtuales del hardware completo. Cada usuario corre su propio SO sobre esa máquina virtual.

Cómo lo voy a presentar (estructura):

1. Historia breve y para qué se usa el SO.
2. Cómo está organizado su kernel, con un diagrama.
3. Ventajas y desventajas de ese diseño.
4. Comparación con los otros tipos (cuadro final).

Comparación rápida (para cerrar la presentación)

Diseño	Rendimiento	Robustez	Facilidad de extender
Monolítico	Alto	Baja (un driver malo afecta todo)	Media
Modular	Alto	Media	Alta (módulos en caliente)
Capas	Medio	Media	Media
Micronúcleo	Menor (por los mensajes)	Alta	Alta
Máquina virtual	Menor (capa extra)	Alta (aislamiento)	Alta

3. Versión 6 (entrega: 19 de octubre de 2026)

3.1 Instalar un nuevo kernel en Ubuntu, Oracle Linux o CentOS

Elijo **Ubuntu** porque es la más simple de probar. Hay dos formas.

Forma A: instalar un kernel ya compilado desde los repositorios (la más segura)

```

uname -r                                # kernel actual
apt search linux-image-generic-hwe      # ver opciones disponibles
sudo apt update
sudo apt install linux-generic-hwe-$(lsb_release -rs)
sudo reboot
uname -r                                # debe mostrar el kernel nuevo

```

1. `uname -r` guarda la versión actual para comparar después.
2. `apt install linux-generic-hwe-...` instala el kernel más reciente que ofrece la versión de Ubuntu.
3. Se reinicia y se vuelve a ejecutar `uname -r` para confirmar el cambio.

Forma B: compilarlo desde el código fuente de kernel.org (la que se parece a la actividad de FreeBSD)

```

sudo apt install build-essential libncurses-dev bison flex libssl-dev libelf-dev bc
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.x.y.tar.xz # poner la
versión real
tar xf linux-6.x.y.tar.xz && cd linux-6.x.y
cp /boot/config-$(uname -r) .config
make olddefconfig
make -j$(nproc)
sudo make modules_install
sudo make install
sudo update-grub && sudo reboot

```

Es el mismo proceso conceptual que en FreeBSD: configurar (`.config`), compilar, instalar y reiniciar. Como aviso: la diapositiva 34 dice que el código fuente pesa cientos de MiB y la compilación es lenta. Conviene hacerlo en una VM con al menos 4 GB de RAM y 30 GB de disco.

En Oracle Linux y CentOS el instalador de paquetes es `dnf`, y el equivalente de `update-grub` es `grub2-mkconfig -o /boot/grub2/grub.cfg`.

3.2 ¿Qué tipo de kernel tienen estos SO?

La lista sale de la diapositiva 40. Esta clasificación es la que se ve en la literatura, pero varía según la fuente en los casos difíciles. Los que marco con (*) son los más discutibles.

Grupo 1: BeOS, Mach, Mac OS X, newOS

SO	Tipo de kernel	Comentario
BeOS	Híbrido (*)	Kernel modular con servicios en espacio de usuario.
Mach	Micronúcleo	Desarrollado en Carnegie Mellon. Es la base de otros sistemas.
Mac OS X	Híbrido	Su kernel XNU combina Mach (micronúcleo) con partes de BSD.
newOS	Híbrido / modular (*)	Es un proyecto pequeño y pocas fuentes lo clasifican con claridad.

Grupo 2: AdeOS, EROS, KeyKOS, BriX-OS

SO	Tipo de kernel	Comentario
AdeOS	Nanonúcleo	Es una capa de virtualización de hardware que permite correr varios SO.
EROS	Nanonúcleo	Se basa en capacidades. Es el sucesor de KeyKOS.
KeyKOS	Nanonúcleo	Sistema de capacidades muy pequeño.
BriX-OS	Ver grupo 3 (*)	BriX aparece de nuevo en el grupo de espacio de direcciones único.

Grupo 3: Opal, Mungi, BriX

Los tres son **SASOS (Single Address Space Operating System)**. Todos los procesos comparten un único espacio de direcciones virtual, y la protección se logra con capacidades en vez de espacios separados.

Grupo 4: MIT exokernel

Es un **exonúcleo**. El kernel casi no abstrae el hardware y solo reparte recursos de forma segura. Las abstracciones las ponen las bibliotecas en espacio de usuario (*LibOS*).

4. Evidencias reales

Entorno de trabajo: todas las pruebas las hice en mi computador con **Windows 11**. Para la parte de Linux usé **Ubuntu 26.04 LTS sobre WSL2**, y para la parte de FreeBSD usé una **máquina virtual de FreeBSD 15.1 en VirtualBox**. Las imágenes de esta sección son capturas de las salidas reales de la terminal de esos dos entornos. Mac OS X aparece en este documento solo como ejemplo teórico de kernel híbrido (sección 3) y no participó en ninguna prueba.

4.1 Ubuntu 26.04 (WSL2): compilación del kernel 6.18.54

Lo hice en Ubuntu 26.04 LTS ejecutándose sobre WSL2 dentro de mi Windows 11, con 12 núcleos y 8 GB de RAM disponibles. Descargué el código fuente de kernel.org, partí de la configuración del kernel que estaba en uso (/proc/config.gz, el equivalente a copiar /boot/config-\$(uname -r)) y compilé la imagen y los módulos.

Kernel antes de empezar y descarga del código fuente:

```
$ uname -r
6.18.33.2-microsoft-standard-WSL2

$ wget linux-6.18.54.tar.xz
2026-09-28 22:30:48 URL:https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.18.54.tar.xz
[154801608/154801608]
-rw-r--r-- 1 sergio_gomez sergio_gomez 148M Sep 25 09:47 linux-6.18.54.tar.xz
```

Configuración (make olddefconfig):

```
#  
# configuration written to .config  
#
```

Compilación (make -j12 bzImage modules): terminó sin errores.

```
real    30m50.256s  
user    293m32.196s  
sys     54m55.183s
```

Resultado:

```
$ make kernelrelease  
6.18.54-microsoft-standard-WSL2  
  
$ ls -lh arch/x86/boot/bzImage  
-rw-r--r-- 1 sergio_gomez sergio_gomez 15M Sep 28 23:01 arch/x86/boot/bzImage  
  
$ file arch/x86/boot/bzImage  
arch/x86/boot/bzImage: Linux kernel x86 boot executable, bzImage, version 6.18.54-microsoft-  
standard-WSL2 (sergio_gomez@DESKTOP-8T2JB98) #1 SMP PREEMPT_DYNAMIC Mon Sep 28 22:59:50  
-05 2026  
  
$ find . -name "*.ko" | wc -l  
960  
  
$ make modules_install INSTALL_MOD_PATH=~/.kernel/rootfs  
rc=0  
6.18.54-microsoft-standard-WSL2  
  
$ modinfo ../kernel/net/nsh/nsh.ko  
license:      GPL v2  
description:  NSH protocol  
author:       Jiri Benc <jbenc@redhat.com>  
intree:       Y
```

Del resultado saco tres cosas: - El kernel nuevo (6.18.54) es un parche más nuevo del que estaba corriendo (6.18.33.2). - La imagen bzImage pesa 15 MB, y los componentes que no van dentro se compilaron como 960 módulos .ko, tal como explica la diapositiva 34. - WSL2 arranca el kernel desde Windows y no desde GRUB, así que el paso update-grub y el reinicio no aplican ahí. En una instalación normal de Ubuntu ese último paso es el que activa el kernel nuevo.

4.2 FreeBSD 15.1: kernel personalizado CLASEKERN

Usé la máquina virtual de FreeBSD 15.1-RELEASE (amd64) que corre en VirtualBox sobre mi Windows 11, con 6 CPU y 4 GB de RAM, y trabajé con ella por SSH desde Windows. Amplié el disco a 30 GB porque el original de 6 GB no alcanzaba para el código fuente y la compilación, y descargué las fuentes (src.txz, 241 MB) con fetch desde download.freebsd.org. Como el sistema es de 64 bits, la ruta de configuración es amd64 y no i386.

Identificar el sistema:

```
# cat /etc/os-release
NAME=FreeBSD
VERSION="15.1-RELEASE"
VERSION_ID="15.1"
ID=freebsd
# freebsd-version -u -k -r
15.1-RELEASE
15.1-RELEASE
15.1-RELEASE
# getconf LONG_BIT
64
# uname -a
FreeBSD freebsd 15.1-RELEASE FreeBSD 15.1-RELEASE releng/15.1-n283562-96841ea08dcf GENERIC
amd64
```

Configurar el kernel (cambié la línea ident a CLASEKERN para poder reconocerlo después):

```
# cd /usr/src/sys/amd64/conf
# mkdir /root/kernels
# cp GENERIC /root/kernels/CLASEKERN
# ln -s /root/kernels/CLASEKERN
lrwxr-xr-x  1 root wheel 23 Sep 28 23:16 CLASEKERN -> /root/kernels/CLASEKERN
# grep ident CLASEKERN
ident          CLASEKERN
```

Compilar (make -j6 buildkernel KERNCONF=CLASEKERN, desde /usr/src):

```
>>> Kernel(s) CLASEKERN built in 3924 seconds, ncpu: 6, make -j6
3933.05 real    17756.49 user    4551.05 sys
```

Primer intento de instalar (make installkernel KERNCONF=CLASEKERN), que falló:

```
ERROR: The kernel at /boot/kernel was installed from packages.
A packaged kernel should never be updated using installkernel;
this will cause the package database to become out of sync with
the live system state. Either uninstall the packaged kernel,
or install this kernel to a different path using INSTKERNNAME.
```

Instalación en otra ruta y arranque:

```
# make installkernel KERNCONF=CLASEKERN INSTKERNNAME=CLASEKERN
>>> Install kernel(s) CLASEKERN completed in 221 seconds, ncpu: 6
# ls -lh /boot/CLASEKERN/kernel
-r--r--r--  1 root wheel  28M Sep 29 00:21 /boot/CLASEKERN/kernel
# nextboot -k CLASEKERN
# init 6
```

Después de reiniciar:

```
# uname -a
FreeBSD freebsd 15.1-RELEASE FreeBSD 15.1-RELEASE CLASEKERN amd64
# uname -i
CLASEKERN
# sysctl kern.bootfile
kern.bootfile: /boot/CLASEKERN/kernel
# sysctl kern.ident
kern.ident: CLASEKERN
# kldstat | head -3
Id Refs Address          Size Name
  1    5 0xffffffff80200000  22fb980 kernel
  2    1 0xffffffff82c18000   3220 intpm.ko
```

Con esto queda comprobado que el sistema arrancó con mi kernel CLASEKERN. Antes decía GENERIC al final de `uname -a` y ahora dice CLASEKERN. La compilación tardó más de una hora aun con 6 CPU, lo que confirma lo que decía la diapositiva 35 sobre que compilar un kernel lleva su tiempo.

5. Lo que aprendí

Lo que más me quedó es que el kernel no es "el sistema operativo" completo, sino la parte que controla CPU, memoria y dispositivos. Cambiar de un kernel monolítico a un micronúcleo es un problema de equilibrio entre **velocidad** y **robustez**. Compilar un kernel propio tiene sentido cuando se necesita quitar lo que sobra o agregar hardware específico. Para lo demás, alcanza con cargar un módulo.